Active Sample Theory of Hard Negative Mining

Unifying Detection HEM and Metric-Learning HNM, with a Gradient-Level Diagnosis of Embedding Collapse

Jihun Jang · 02521122 · UST / ETRI · Advanced Study 02 · April 2026

Abstract. Detection HEM and deep-metric HNM share one principle — feed gradient only to samples with L > 0 — yet only the latter collapses under aggressive mining. From $\partial L/\partial a = 2(n-p)$ the per-triplet gradient magnitude equals $\|p-n\|$, so hard mining actually *shrinks* the gradient it is supposed to amplify. A NumPy-only experiment (10 classes, triplet loss, 3 strategies × 2 seeds × 30 epochs) confirms: hard achieves 100% active ratio, 100× smaller $\|p-n\|$, joint intra / inter collapse, and R@1 0.48 → 0.13. The divergence is not mining but *loss structure*: relational supervision has a trivial minimum (all points merged); absolute supervision (CE + labels, as in my fall-detection detector) does not.

1. Research Question

Detection HEM and metric-learning HNM both act on samples with non-zero loss, yet detection tolerates aggressive mining while metric learning collapses. **What gradient-level mechanism produces this asymmetry?** I attack the question along two quantities: the per-triplet gradient proxy $\|p-n\|$ and the joint evolution of (intra-class std, inter-class distance, embedding rank).

2. Motivation

- **Lecture.** Direct extension of the ranking-loss / HNM thread.
- **My paper.** Indoor-CCTV fall detector: miss-based outdoor HEM raised outdoor F1 0.390 → 0.844 with no collapse. This report diagnoses why that tap was safe.
- **VLM / VLAM.** CLIP, SigLIP, NegCLIP, DPO are relational-supervision losses whose central engineering concern is negative mining; the theory transfers directly.

3. Background

Triplet loss on the sphere. With $f_\theta: X \to S^{d-1}$ and squared Euclidean distance, `L = max(0, d(a,p) − d(a,n) + m)`. Supervision is *relational* — only same-/different-class pairs are known.

Active-sample principle. A triplet is active iff the hinge opens. Differentiating gives $\partial L/\partial a = 2(n-p)$ (active) / 0 (inactive). Two consequences: the anchor update direction is purely $n-p$, and its magnitude is $\|p-n\|$ — not a function of how "hard" the negative is.

Mining strategies. *Random* uniform / *semi-hard* d(a,p) < d(a,n) < d(a,p)+m (Schroff 2015) / *hard* argmin_n d(a,n). **Detection HEM** (OHEM; my paper) collects samples with large CE loss — same active principle over *absolute* supervision (each sample has its own one-hot target).

4. Main Analysis

- **Claim 1 — Unification.** Triplet HNM and detection HEM are the same principle (gradient flows only where L > 0) applied to different loss families.
- **Claim 2 — Hard paradox.** Hard mining places n next to a, which is also near p, so $\|p-n\| \to 0$. High active ratio, *small* per-triplet gradient, high variance. Random / semi-hard trade fewer actives for larger, steadier gradients.
- **Claim 3 — Collapse is relational-supervision specific.** Triplet loss is invariant under mapping every input to one vector c (d=0, L=m, $\|p-n\|=0$); hard mining is an efficient search for this minimum. CE + label forbids the trivial minimum: each sample pins a distinct optimum.
- **Hypotheses. H1** $\text{active}_{\text{hard}} > \text{active}_{\text{semi}} > \text{active}_{\text{rand}}$. **H2** $\|p-n\|_{\text{hard}} \ll$ others. **H3** hard → joint intra/inter ↓ collapse. **H4** $R@1_{\text{hard}} \ll$ others.

5. Experimental Verification

Setup. 10-class 64-D Gaussians (σ=0.55), 400 train + 150 test / class. MLP 64→128→64→16 with L2-norm, Adam lr 3e-3, margin 0.2, PK batches (P=K=8), 30 epochs × 2 seeds × 3 strategies. NumPy only, ≈ 20 s wall-clock. Synthetic data are adequate because the question is *structural* (gradient-level), not about absolute recognition accuracy.

Metric	Random	Semi-hard	Hard
Active ratio	0.193	0.315	1.000
‖p−n‖ (mean)	0.812	1.174	0.008
Intra std	0.129	0.213	0.002
Inter dist	0.480	0.718	0.001
R@1	0.478	0.487	0.130

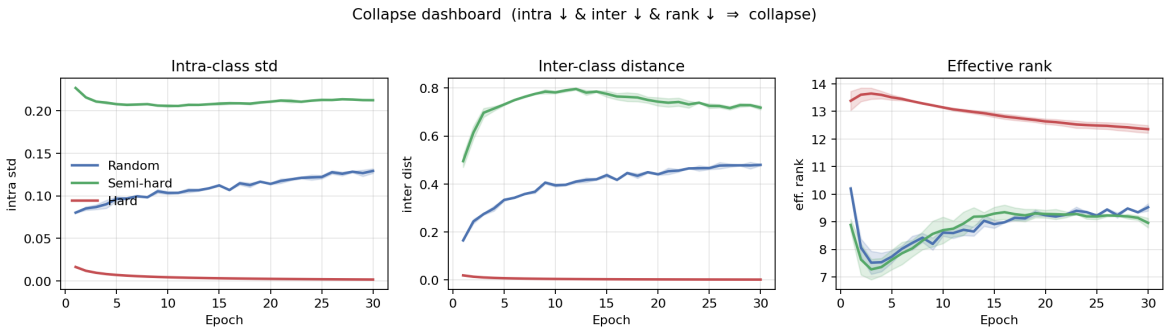

Figure 1. **Collapse dashboard.** Hard mining drags intra and inter distances to zero *jointly*. Semi-hard gives the largest inter-class separation.

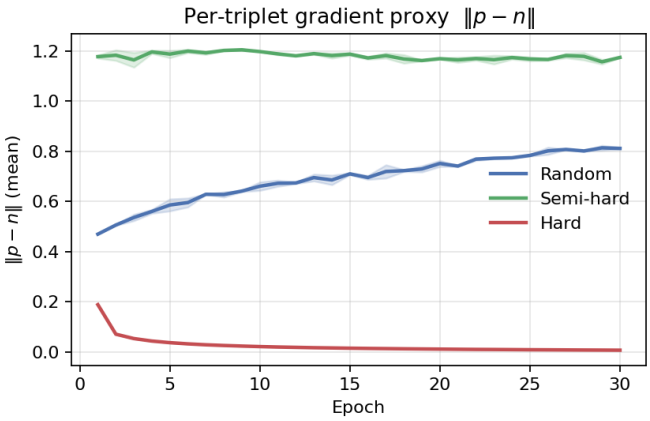

Figure 2. **‖p−n‖.** Hard mining drives it ~100× below the other two — the hard paradox quantified.

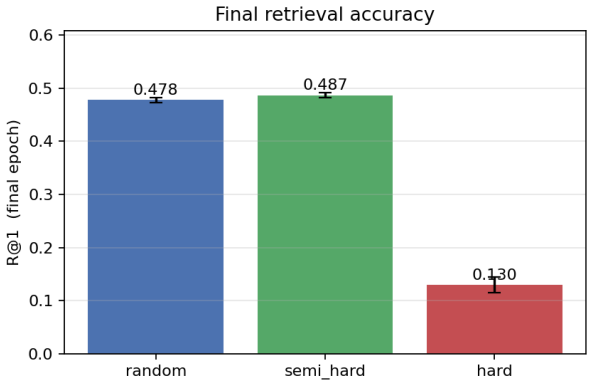

Figure 3. **R@1** collapses to 0.13 for hard; random / semi-hard ≈ 0.48.

6. Interpretation

- **Paradox.** ‖p−n‖ ≈ 0.008 (hard) vs. 0.81 (random) / 1.17 (semi-hard). Higher active population does not compensate for a 100× per-sample signal drop: “hard = big gradient” is empirically wrong.

- **Effective-rank pitfall.** Hard's effective rank (12.5) > random's (9.5) despite clear collapse — once points merge, residual isotropic noise inflates entropy. Effective rank alone is unreliable; read it with intra / inter distances.
- **Random works here only.** 10 isotropic Gaussians in 64-D are largely separable; random negatives are often already informative. On fine-grained data the classical semi-hard > random gap is expected to return.
- **Back to my detector.** CE + labels + regressed boxes has no translation/scale trivial minimum, so miss-based HEM is safe; the same tap on relational supervision lands on collapse.

7. Conclusion

One active-sample principle, two regimes. Absolute supervision (detection CE) can be mined aggressively; relational supervision (triplet, contrastive) cannot, because the loss admits a collapse minimum that hard mining finds efficiently. All four hypotheses were confirmed: hard mining maximized active ratio, *minimized* per-triplet gradient, collapsed intra and inter jointly, and halved R@1. For VLM / VLAM — relational by construction — hard-negative strategies must be paired with a collapse-preventing anchor (normalization, stop-gradient, supervised head, absolute target) *before* being pushed to their limit.

8. Appendix — Program Code

NumPy-only implementation. Hyperparameters: INPUT=64, HIDDEN=(128, 64), EMB=16, 10 classes, 400 train + 150 test / class, σ=0.55, margin=0.2, lr=3e-3, Adam, PK batch (P=K=8), 30 epochs × 2 seeds × 3 strategies, ≈ 20 s wall-clock.

```
import numpy as np

# ---------- model: MLP 64 -> 128 -> 64 -> 16 with L2-normalized output
class MLP:
    def __init__(self, rng):
        he = lambda i, o: rng.standard_normal((i, o)).astype(np.float32) * np.sqrt(2.0/i)
        self.W1, self.b1 = he(64, 128), np.zeros(128, np.float32)
        self.W2, self.b2 = he(128, 64), np.zeros(64, np.float32)
        self.W3, self.b3 = he(64, 16), np.zeros(16, np.float32)
    def params(self): return [self.W1,self.b1,self.W2,self.b2,self.W3,self.b3]
    def forward(self, X):
        Z1 = X @ self.W1 + self.b1; A1 = np.maximum(Z1, 0)
        Z2 = A1 @ self.W2 + self.b2; A2 = np.maximum(Z2, 0)
        Z3 = A2 @ self.W3 + self.b3
        norm = np.linalg.norm(Z3, axis=1, keepdims=True) + 1e-8
        E = Z3 / norm
        return E, (X, Z1, A1, Z2, A2, Z3, norm, E)
    def backward(self, dE, cache):
        X, Z1, A1, Z2, A2, Z3, norm, E = cache
        # L2-normalize Jacobian: if e = z/||z||, then dz = (de - (de.e) e) / ||z||
        dZ3 = (dE - (dE * E).sum(axis=1, keepdims=True) * E) / norm
        dW3, db3 = A2.T @ dZ3, dZ3.sum(0)
        dZ2 = (dZ3 @ self.W3.T) * (Z2 > 0)
        dW2, db2 = A1.T @ dZ2, dZ2.sum(0)
        dZ1 = (dZ2 @ self.W2.T) * (Z1 > 0)
        dW1, db1 = X.T @ dZ1, dZ1.sum(0)
        return [dW1, db1, dW2, db2, dW3, db3]

# ---------- triplet mining (random / semi-hard / hard)
def mine_triplets(E, Y, strategy, margin, rng):
```

```
    B = E.shape[0]
    D = np.maximum(2.0 - 2.0 * E @ E.T, 0.0) # squared dist on sphere
    same = Y[:, None] == Y[None, :]
    pos_mask = same & ~np.eye(B, dtype=bool)
    neg_mask = ~same
    a_l, p_l, n_l = [], [], []
    for a in range(B):
        pi, ni = np.where(pos_mask[a])[0], np.where(neg_mask[a])[0]
        if len(pi) == 0 or len(ni) == 0: continue
        p = rng.choice(pi); dap = D[a, p]
        if strategy == "random": n = rng.choice(ni)
        elif strategy == "hard": n = ni[np.argmin(D[a, ni])]
        elif strategy == "semi_hard":
            dan = D[a, ni]
            cand = ni[(dan > dap) & (dan < dap + margin)]
            if len(cand): n = rng.choice(cand)
            else:
                further = ni[dan > dap]
                n = further[np.argmin(D[a, further])] if len(further) else ni[np.argmax(dan)]
        a_l.append(a); p_l.append(p); n_l.append(n)
    return np.array(a_l), np.array(p_l), np.array(n_l)

# ---------- triplet loss + gradient + observables
# Active rule: L = max(0, d(a,p) - d(a,n) + m). Anchor gradient = 2(n - p) when active.
def triplet_forward_backward(E, a_idx, p_idx, n_idx, margin):
    Ea, Ep, En = E[a_idx], E[p_idx], E[n_idx]
    dap = ((Ea - Ep) ** 2).sum(1); dan = ((Ea - En) ** 2).sum(1)
    raw = dap - dan + margin
    active = raw > 0
    pn_norm = np.linalg.norm(Ep - En, axis=1) # per-triplet gradient proxy
    dE = np.zeros_like(E)
    if active.any():
        m = active.astype(np.float32)[:, None]
        np.add.at(dE, a_idx, m * 2.0 * (En - Ep)) # ∂L/∂a = 2(n - p)
        np.add.at(dE, p_idx, m * 2.0 * (Ep - Ea))
        np.add.at(dE, n_idx, m * 2.0 * (Ea - En))
    dE /= max(len(a_idx), 1)
    return float(np.maximum(raw, 0).mean()), dE, active, pn_norm

# ---------- collapse observables
def collapse_stats(E, Y):
    classes = np.unique(Y)
    centroids = np.stack([E[Y == c].mean(0) for c in classes])
    intra = float(np.mean([E[Y == c].std(0).mean() for c in classes]))
    diffs = centroids[:, None, :] - centroids[None, :, :]
    inter = float(np.sqrt((diffs ** 2).sum(-1))[np.triu_indices(len(classes), 1)].mean())
    s = np.linalg.svd(E - E.mean(0, keepdims=True), compute_uv=False)
    p = s ** 2 / ((s ** 2).sum() + 1e-12)
    eff_rank = float(np.exp(-(p * np.log(p + 1e-12)).sum()))
    return intra, inter, eff_rank
```

```
# ---------- training step (per epoch, per batch)
# for each PK batch:
# E, cache = model.forward(Xb)
# a, p, n = mine_triplets(E, Yb, strategy, margin, rng)
# loss, dE, *_ = triplet_forward_backward(E, a, p, n, margin)
# grads = model.backward(dE, cache)
# adam.step(model.params(), grads)
```

AI assistance. This report was developed through tutoring and discussion with an AI assistant. The AI acted as a Socratic tutor — posing questions, surfacing candidate framings, and pushing back on my initial intuitions — across topic scoping, gradient derivation, collapse mechanism, observable design, and writing.